Date: 25/8/25

# Toc H Institute of Science & Technology

(An ISO 9001:2015 Certified Institution)

Arakkunnam

## INTERNAL ASSESSMENT MAIN ANSWER BOOKLET

| | I | II | III | IV | V | VI | VII | VIII | IX | X | XI | XII | XIII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | 3 | 3 | ½ | 3 | ½ | 6 | 5 | 3 | | | | | |
| b | | | | | | 8 | 9 | | | | | | |
| c | | | | | | | | | | | | | |
| d | | | | | | | | | | | | | |
| e | | | | | | | | | | | | | |
| f | | | | | | | | | | | | | |
| g | | | | | | | | | | | | | |
| h | | | | | | | | | | | | | |
| Total | | | | | | | | | | | | | |
| Grand Total | | | | | | | | | | | | | |

TOTAL 41½/50

[redacted]

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Name | JOHN JOSEPH | | | | | | | | | | |
| Branch & Semester | CS S5 | | | | | | | | | | |
| Subject with Code | CST 305 SYSTEM SOFTWARE | | | | | | | | | | |
| Roll No. | 4 | University Reg. No: | T | O | C | 2 | 3 | C | S | 0 | 6 |
| Signature of Invigilator | | No. of Additional Sheets | | | | | | | | | |

# PART - A

1.

| System Software | Application Software |
|---|---|
| * used to interact with hardware | * Used to perform a particular task |
| * system dependent | * system independent |
| * installs when OS installs | * installs according to user needs |
| * User does not interact with system software | * user interact with application software |
| * can run independently | * can't run independently, requires a OS |
| * eg: linker, loader, text editor | * eg: MS office packages, media player etc.. |

2. Three assembler directives used in SIC are

START : It specifies the name & starting address of the program

END : It indicates the end of the program and address of first executable instruction.

BYTE : It generates hexadecimal / character constants

WORD : It generates integer constants.

Assembler directives are the instruction given to the assembler for executing a program.

3. Format of an object program generated by 2 pass assembler is

START

. . HEADER

END

4

```
    LDA   ALPHA
    STA   GAMMA
    LDA   BETA
    STA   ALPHA
    LDA   GAMMA
    STA   ALPHA

ALPHA   RESW  1
BETA    RESW  1
GAMMA   RESW  1
```

~~RMO S,A~~

~~This is format~~

PART-B

6a. Linker

* Linking is the process of collecting & combining various pieces of code into a single file and ~~to~~ load into memory for execution.
* Linking can be done during compile time and run time
* Large programs are decomposed and divided into small modules for easy compiling. If any changes has to be made in a module we can compile it seperately and relink again instead of compiling entire program.

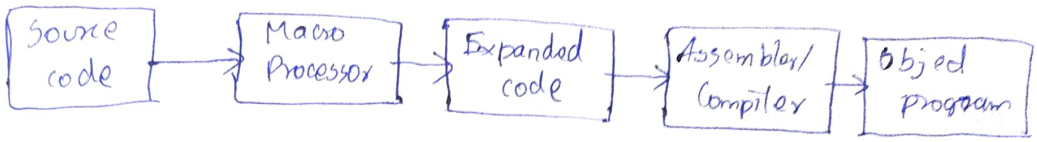
Loader

* Loader is a utility of ~~system s~~ OS
* It loads programs from secondary memory to main memory for ~~it~~ execution.
* loader executes without user interaction. It

memory space for the program on its own.

## Macro Processor

* Macros are the special code segments which are ~~ones~~ defined in program and can be called ~~repeatedly~~ repeatedly from different ~~pa~~ places of the program.
* Macros help the programmer to write a shorthand version of the program.




## Text Editor

* Text editor is a computer program that edits plain text
* Text editor is a program that allows user to create source in the form of text into main memory.
* It allows creation, deletion, updation of text

6b

## SIC/XE

It stands for Simplified Instructional Computer Extended Version.

### 1. Memory

* It has $2^{20}$ bytes of memory available (1 Mb)
* Each word is of length of 24 bits
* Three consecutive bytes forms a word.
* Each byte has 8 bits
* All the memory address are byte addressible.

### 2. Register

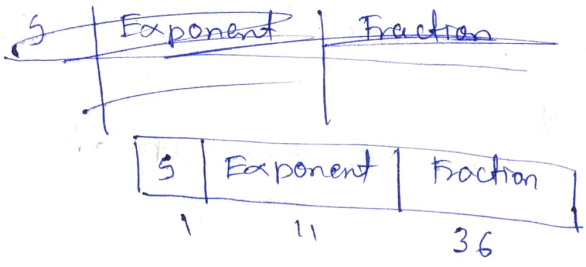
* There are 9 registers in SIC/XE architecture.

| Mnemonic | Number | Use |
|---|---|---|
| A | 0 | Accumulator for arithmetic operation |

| | | |
|---|---|---|
| X | 1 | Index register, for addressing |
| L | 2 | Linkage register, JSUB |
| PC | 8 | Program counter, next instruction to be executed |
| SW | 9 | Status word, including CC |
| B | 3 | Base register |
| S | 4 | General purpose register |
| T | 5 | General purpose register |
| F | 6 | Floating point accumulator (48 bit) |

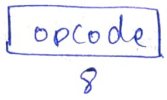
3. Data Format

* One word for integer representation
* One byte for character representation
* 2's complement for representing negative values
* Floating point numbers are represented using 48 bit

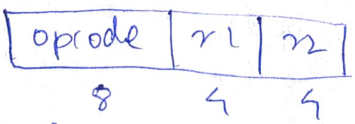



4. Instruction Format

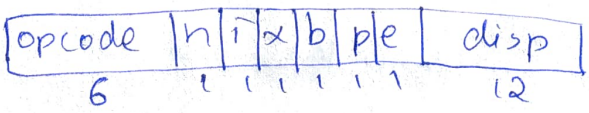
* Format 1 : contains only opcode



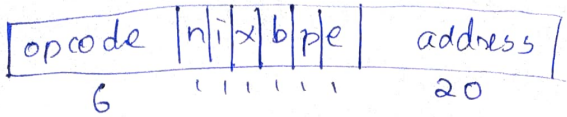

* Format 2: contains first 8 bit opcode, next four for register 1, next four for register 2




* Format 3: first 6 bit opcode, next 6 bit flag and remaining 12 bit for displacement




Format 4: first 6 bit opcode, next 6 bit flag and 20 bit for address




## 5. Addressing Modes

* Base ~~flat~~ relative
  b = 1, p = 0 TA = disp + [B]
* PC relative
  b = 0, p = 1 TA = disp + [PC]
* direct addressing
  b = 0, p = 0, x = 0 TA = disp / address
* with indexing
* PC relative with indexing
  b = 0, p = 1, x = 1 TA = disp + [PC] + [X]
* Base relative with indexing
  b = 1, p = 0, x = 1 TA = disp + [B] + [X]
* ~~Immedia~~ Immediate addressing
  n = 0, i = 1
  # symbol is used to indicate immediate addressing
* Indirect addressing
  n = 1, i = 0
  @ symbol is used to indicate indirect addressing

## 6. Instruction set

* Data Move instruction - LDA, LDX, STA
* Arithmetic instruction - ADD, SUB
* Compare instruction - <, >, =
* Jump instruction - JLT, JEQ
* Subroutine linkage instruction - JSUB, RSUB
* Floating point arithmetic operation - ADDF, SUBF
* register to register arithmetic operation - ADDR, SUBR

* register move instruction - RMO, r1, r2

2. Input & Output

* TD - Test Device : it indicates whether the device is ready to ~~acc~~ transmit or receive data
* WD, RD - ~~Rea~~ It is used to read and write data
* ~~El~~ SIC/XE uses 10 channels. These includes SIO, TIO & HIO for start, test, & halt of IO channels.

7a. ALPHA = 4 * BETA - 9

```
LDA BETA
LDS #4
MULR S,A
SUB #9
STA ALPHA

BETA RESW 1
ALPHA RESW 1
```

b

Pass 1

```
begin
  read ~~first~~ input line
  if OPCODE = 'START' then
    begin
      save #[operand] as starting address
      initialize LOCCTR ~~to~~ as starting address
      write line to intermediate file
      read next input line
    end{if START}
  else
    initialize LOCCTR to 0
  while ~~8~~'OPCODE ≠ END' do
    ~~if~~ begin
      if there is no comment line then
        begin
```

```
if there is a symbol in LABEL field then
    SEARCH SYMTAB FOR LABEL
    if found then
        set error flag (duplicate symbol)
    else
        store (LOCCTR, LABEL) to the SYMTAB
    end (if symbol)
    SEARCH OPTAB for OPCODE
    if found then
        add 3 [instruction length] to LOCCTR
    else if 'OPCODE = WORD' then
        add 3 to LOCCTR
    else if 'OPCODE = RESW' then
        add 3 * #[operand] to LOCCTR
    else if 'OPCODE = 'RESB' then
        add #[operand] to LOCCTR
    else if 'OPCODE = 'BYTE' then
    begin
        find length of constant in bytes
        add length to LOCCTR
    end (if BYTE)
    else
        set error ~~code~~ flag (invalid operation code)
    end (if no comment line)
    write line to intermediate file
    read next input line
end (while not END)
    ~~write~~ write last to intermediate file
    save [LOCCTR - starting address] as program leng
end (Pass 1)
```

* Data structures used in Pass 1 are
  - SYMTAB
  - OPTAB
* Assembly language and optab is given as input
* SYMTAB contains all the labels and their values
* It is used in Pass 2 to replace labels with values
* OPTAB contains all the OPCODE and comment along with instruction length

4

| LOCCTR | LABEL | Opcode | Operand | LOCCTR after |
|---|---|---|---|---|
| 4000 | TEST | START | 4000 | 4000 |
| 4000 | FIRST | LDA | FIVE | 4003 |
| 4003 | | STA | ALPHA | 4006 |
| 4006 | ALPHA | RESW | 2 | 400C |
| 400C | FIVE | WORD | 5 | 400F |

Length of program = LOCCTR - starting address

~~= 4000 - 400F~~

= 400F - 4000

= F = 16 bit

SYMTAB

| | |
|---|---|
| LDA | 4003 |
| STA | 4006 |
| RESW | 400C |
| WORD | 400F |

5 RMO A, A

This is format 2

| opcode | n1 | n2 |
|---|---|---|
| 8 | 4 | 4 |

A → 0000

RMO → 68 → 0110 1000

0110 1000 0000 0000

6 8 0 0

LDA #1

This is format 3

disp = 001

opcode = 00

| opcode | n | i | x | b | p | e | disp |
|---|---|---|---|---|---|---|---|
| 6 | 1 | 1 | 1 | 1 | 1 | 1 | 12 |

000000 010000 001

0 10 001

-JSUB RDREC

This is format 4

address = 01036

opcode → 48 → 0100 1000

| opcode | n | i | x | b | p | e | address |
|---|---|---|---|---|---|---|---|
| 6 | 1 | 1 | 1 | 1 | 1 | 1 | 20 |

0100 1011 1001 01036

4